Lecture IV: Scheduling

Programming: Everyday Decision-Making Algorithms

LECTURER
Dr. Nils Roemer, External Lecturer

AFFILIATION
Planningio GmbH

Today in one sentence

There is no single best order for your tasks: first decide what you want to achieve, and the right rule follows.

Why it matters

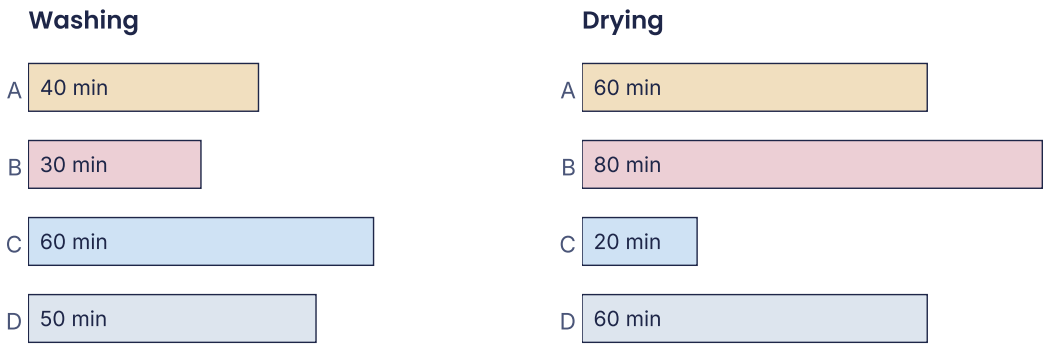
Monday morning, ten tasks on your list: where do you start? Companies face the same question at scale: which order on the production line, which customer order first, which project gets the team this week.

Roadmap

1. The basics: when order matters, and when it doesn't
2. Your goal decides the rule: deadlines, waiting time, value
3. Real life: dependencies, interruptions and thrashing

The basics: when order matters, and when it doesn't

Four loads of laundry, one washer, one dryer



Goal: all laundry done as early as possible

Task	Washing	Drying
A	40min	60min
B	30min	80min
C	60min	20min
D	50min	60min

Goal: **Minimize total time for washing and drying all loads**

Question: **An idea how to solve this?**

Johnson's rule: shortest job first or last

Rule: To find the optimal solution:

1. Find the job with **shortest duration**:
   - If on **Machine 1** → Schedule First
   - If on **Machine 2** → Schedule Last
   - If **equal** → Choose randomly
2. Remove job from list and **repeat**

Applying Johnson's Rule

Task	Washing	Drying	Schedule
A	40min	60min	
B	30min	80min	
C	60min	20min	
D	50min	60min	

Question: **What's the first scheduled task?**

Applying Johnson's Rule

Task	Washing	Drying	Schedule
A	40min	60min	
B	30min	80min	
C	60min	20min	4
D	50min	60min	

- **In Task C**, the dryer is the **shortest** task.
- It is on **Machine 2** → **Schedule Last**

Question: **What's the next task?**

Applying Johnson's Rule

Task	Washing	Drying	Schedule
A	40min	60min	
B	30min	80min	1
C	60min	20min	4
D	50min	60min	

- **In Task B**, the washing machine is the **shortest** task.
- It is on **Machine 1** → **Schedule First**

Question: **What's the next task?**

Applying Johnson's Rule

Task	Washing	Drying	Schedule
A	40min	60min	2
B	30min	80min	1
C	60min	20min	4
D	50min	60min	

- **In Task A**, the washing machine is the **shortest** task.
- It is on **Machine 1** → **Schedule Second**

Now, we only have one task left!

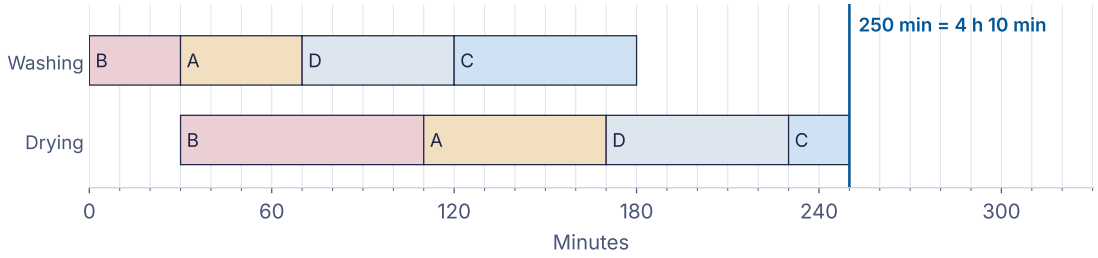
Applying Johnson's Rule

Task	Washing	Drying	Schedule
A	40min	60min	2
B	30min	80min	1
C	60min	20min	4
D	50min	60min	3

Final sequence: B A D C

Optimal Solution

Optimal Solution: B A D C

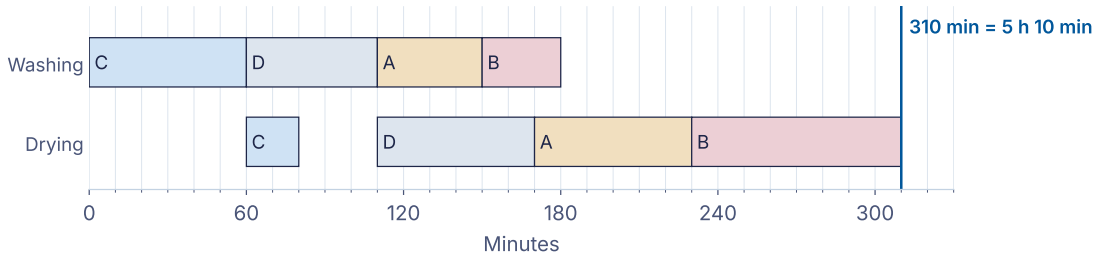


Total time: 4 hours 10 minutes

Question: **Is there a worse solution?**

Suboptimal Solution

Suboptimal Solution: C D A B



Total time: 5 hours 10 minutes

Question: **What's the difference?**

Industrialization

- First systematic visualization by Frederick Taylor
- Henry Gantt develops the Gantt Chart around 1910
- Key tool of industrial production planning
- But no scheduling theory yet!

Question: **Who knows what a Gantt Chart is?**

Modern Scheduling Theory

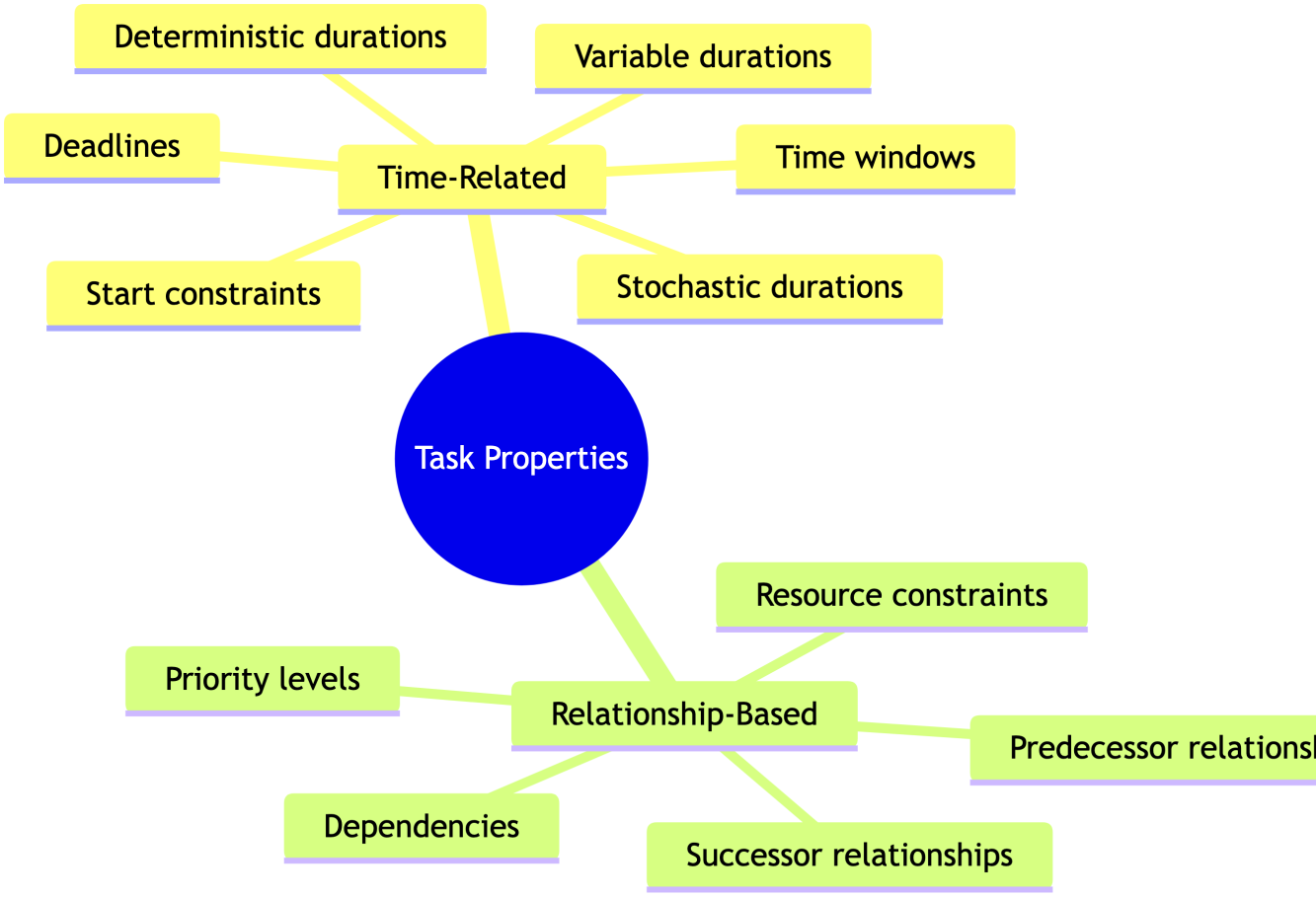
- RAND Corporation founded (1948)
- Selmer Johnson publishes Johnson's Rule in 1954
- Beginning of modern scheduling theory
- Many more algorithms and methods developed since

Scheduling is everywhere in professional practice

- **Project management:** task dependencies, resource allocation
- **Software:** CPU, threads, job queues
- **Operations:** production, supply chain
- **Healthcare:** patients, surgeries, staff
- **Transportation:** routes, crews, maintenance

Tasks differ in time and in relationships

Question: **What properties can scheduled tasks have?**



Which tasks do you deal with?

Question: **What types of tasks do you deal with most often?**

One person instead of two machines

One person

A 40 min

B 30 min

C 60 min

D 50 min

Question: **What is different from before?**

For total time alone, the order doesn’t matter

> ℹ Note
>
> With one person and no deadlines, every order takes the same **total time**.

Question: **But is it that simple?**

> ! Important
>
> Order becomes crucial with **deadlines, priorities and dependencies**.

The basics: when order matters, and when it doesn’t

What we covered

Sometimes the order of tasks changes the result, sometimes it doesn’t. Knowing which saves time.

Your goal decides the rule: deadlines, waiting time, value

Roadmap

1. The basics: when order matters, and when it doesn’t
2. Your goal decides the rule: deadlines, waiting time, value
3. Real life: dependencies, interruptions and thrashing

Goal 1: nobody waits too long past the deadline

Tasks with individual deadlines:

```
Task  Duration  Deadline
  A     40min    110min
  B     30min     90min
  C     60min    150min
  D     50min     70min
  E     30min    210min
```

Goal: **Minimize maximum deadline violation**

Question: **An idea how to solve this?**

Earliest Due Date: sort by deadline

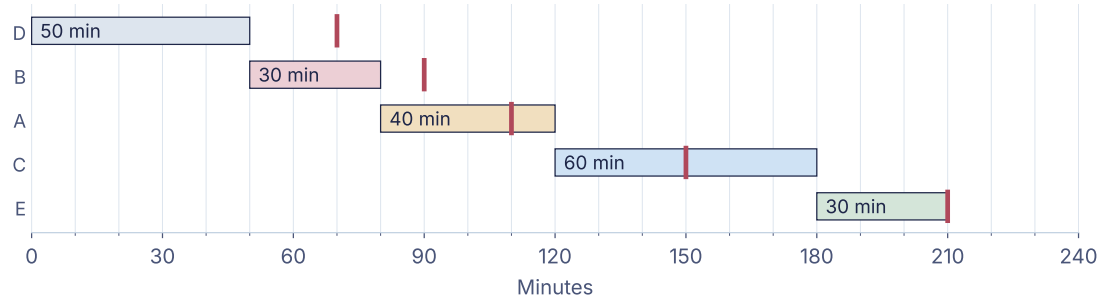
Rule: Sort the tasks by deadline (**EDD**).

```
Task  Duration  Deadline
  A     40min    110min
  B     30min     90min
  C     60min    150min
  D     50min     70min
  E     30min    210min
```

```
Task  Duration  Deadline
  D     50min     70min
  B     30min     90min
  A     40min    110min
  C     60min    150min
  E     30min    210min
```

Let's visualize this!

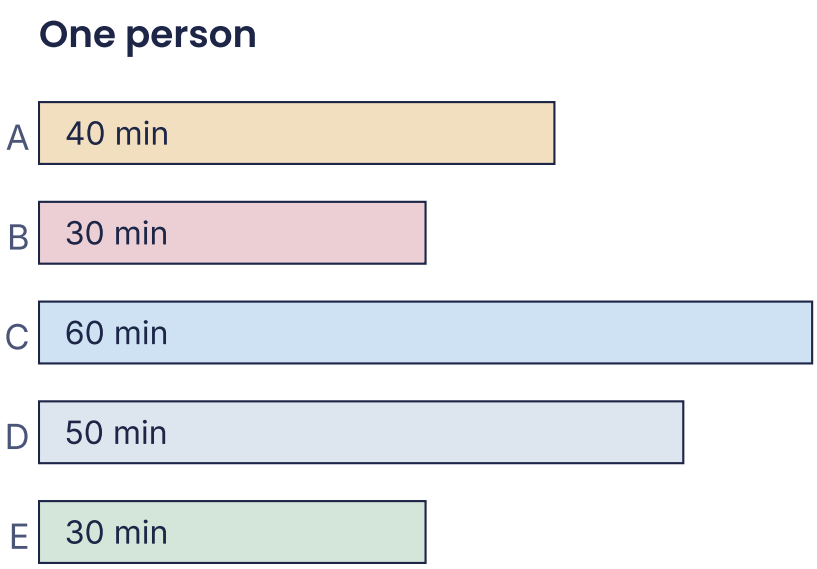
The EDD schedule



Red marks: deadlines.

Question: **What's the maximum delay here?**

Goal 2: as little waiting as possible



Instead of deadlines, we now have **processing times.**

Goal: Min. total waiting time

Question: **Any ideas?**

Rule: Always schedule the shortest remaining task

Shortest Processing Time: shortest task first

Rule: Always schedule the **shortest remaining task**. Choose random if multiple tasks are tied.

Task	Duration	Schedule
A	40min	
B	30min	
C	60min	
D	50min	
E	30min	

Question: **What's the order of scheduled tasks?**

Shortest Processing Time: shortest task first

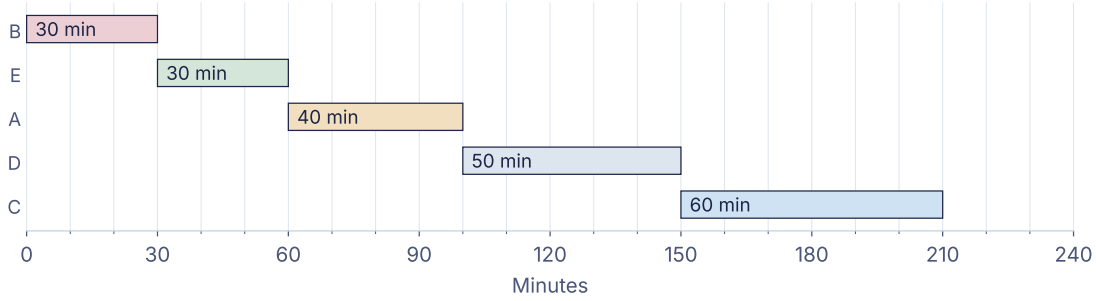
Rule: Always schedule the **shortest remaining task**. Choose random if multiple tasks are tied.

Task	Duration	Schedule
A	40min	3
B	30min	1
C	60min	5

```
D    50min        4
E    30min        2
```

Final sequence: B E A D C or E B A D C

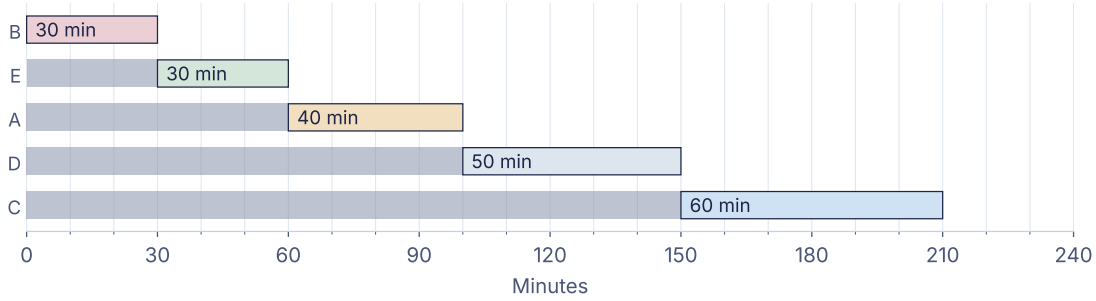
The SPT schedule



Question: **Where can we see the waiting time?**

Waiting time adds up to 340 minutes

Total waiting time: 340 minutes



Grey: time a task waits before it starts.

Question: **Would this be applicable for your work?**

Goal 3: deliver the most value

- Change: Tasks now have **different values**, e.g. revenue for a consultant

```
Task  Duration  Revenue
  A    20min    €240
  B    30min    €200
  C    60min    €120
  D    50min    €70
  E    30min    €130
  F    40min    €120
  G    20min    €100
  H    70min    €110
  I    50min    €90
```

Question: **Any ideas how to approach this?**

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	
B	30min	€200	6.7	
C	60min	€120	2.0	
D	50min	€70	1.4	
E	30min	€130	4.3	
F	40min	€120	3.0	
G	20min	€100	5.0	
H	70min	€110	1.6	
I	50min	€90	1.8	

Question: **What's the order of scheduled tasks?**

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	
C	60min	€120	2.0	
D	50min	€70	1.4	
E	30min	€130	4.3	
F	40min	€120	3.0	
G	20min	€100	5.0	
H	70min	€110	1.6	
I	50min	€90	1.8	

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	
D	50min	€70	1.4	
E	30min	€130	4.3	
F	40min	€120	3.0	
G	20min	€100	5.0	
H	70min	€110	1.6	
I	50min	€90	1.8	

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	
D	50min	€70	1.4	
E	30min	€130	4.3	
F	40min	€120	3.0	
G	20min	€100	5.0	3
H	70min	€110	1.6	
I	50min	€90	1.8	

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	
D	50min	€70	1.4	
E	30min	€130	4.3	4
F	40min	€120	3.0	
G	20min	€100	5.0	3
H	70min	€110	1.6	
I	50min	€90	1.8	

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	
D	50min	€70	1.4	
E	30min	€130	4.3	4
F	40min	€120	3.0	5
G	20min	€100	5.0	3
H	70min	€110	1.6	
I	50min	€90	1.8	

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	6
D	50min	€70	1.4	
E	30min	€130	4.3	4
F	40min	€120	3.0	5
G	20min	€100	5.0	3
H	70min	€110	1.6	
I	50min	€90	1.8	

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	6
D	50min	€70	1.4	
E	30min	€130	4.3	4
F	40min	€120	3.0	5
G	20min	€100	5.0	3
H	70min	€110	1.6	
I	50min	€90	1.8	7

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2
C	60min	€120	2.0	6
D	50min	€70	1.4	
E	30min	€130	4.3	4
F	40min	€120	3.0	5
G	20min	€100	5.0	3
H	70min	€110	1.6	8
I	50min	€90	1.8	7

Weighted SPT: highest value per minute first

Rule: Schedule by revenue per minute (descending)

Task	Duration	Revenue	Revenue/Min	Schedule
A	20min	€240	12.0	1
B	30min	€200	6.7	2

```
  C    60min    €120      2.0          6
  D    50min    €70       1.4          9
  E    30min    €130      4.3          4
  F    40min    €120      3.0          5
  G    20min    €100      5.0          3
  H    70min    €110      1.6          8
  I    50min    €90       1.8          7
```

> Metric Priorities
>
> Without revenues, we can use the same approach with **metric priorities**!

Deadlines with dependencies

Question: **What was earliest due date again?**

- Sort the tasks by deadline
- Now some tasks must **wait for a predecessor**

```
Task  Duration  Deadline  Predecessor
  A    40min    110min    None
  B    30min     90min    D
  C    60min    150min    A
  D    50min     70min    None
  E    30min    210min    C
```

Question: **Any ideas how to solve this?**

Lawler's algorithm: fill the schedule from the back

Rule: We can use **Lawler's Algorithm** (1968)

1. Consider all tasks without successors
2. Choose the one with **latest deadline**
3. Schedule the task **last**
4. **Remove** it from the network and start again

Lawler's Applied

```
Task  Duration  Deadline  Predecessor Schedule
  A    40min    110min    None
  B    30min     90min    D
  C    60min    150min    A
  D    50min     70min    None
  E    30min    210min    C
```

Question: **What's the schedule?**

Lawler’s Applied

Task	Duration	Deadline	Predecessor	Schedule
A	40min	110min	None	
B	30min	90min	D	
C	60min	150min	A	
D	50min	70min	None	
E	30min	210min	C	5

Lawler’s Applied

Task	Duration	Deadline	Predecessor	Schedule
A	40min	110min	None	
B	30min	90min	D	
C	60min	150min	A	4
D	50min	70min	None	
E	30min	210min	C	5

Lawler’s Applied

Task	Duration	Deadline	Predecessor	Schedule
A	40min	110min	None	3
B	30min	90min	D	
C	60min	150min	A	4
D	50min	70min	None	
E	30min	210min	C	5

Lawler’s Applied

Task	Duration	Deadline	Predecessor	Schedule
A	40min	110min	None	3
B	30min	90min	D	2
C	60min	150min	A	4
D	50min	70min	None	
E	30min	210min	C	5

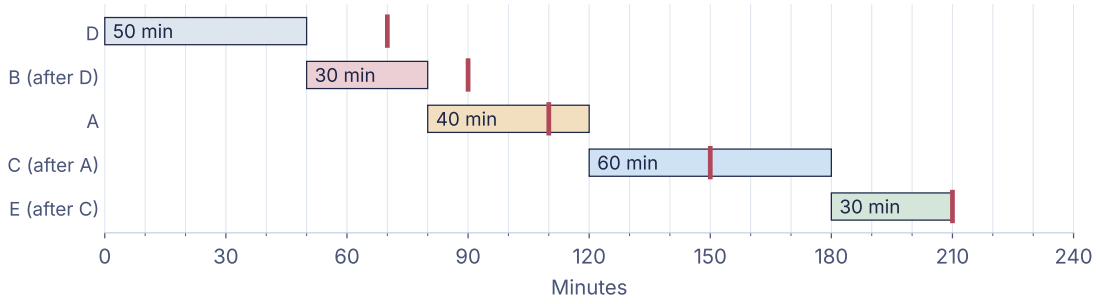
Lawler’s Applied

Task	Duration	Deadline	Predecessor	Schedule
A	40min	110min	None	3
B	30min	90min	D	2
C	60min	150min	A	4
D	50min	70min	None	1
E	30min	210min	C	5

Successor Tasks

Note, how all tasks become tasks without successors at some point.

The Lawler schedule



The task in brackets must be finished first. Red marks: deadlines.

Question: **What's the maximum delay?**

Waiting time with dependencies has no fast exact solution

- NP-hard: no known algorithm solves it efficiently
- True for **most scheduling problems**
- In practice we use **heuristics**, good rules without a guarantee

Your goal decides the rule: deadlines, waiting time, value

What we covered

Each goal has its own simple rule: earliest deadline first to avoid being late, shortest task first to cut waiting, highest value per minute first to maximize what you deliver.

Real life: dependencies, interruptions and thrashing

Roadmap

1. The basics: when order matters, and when it doesn't
2. Your goal decides the rule: deadlines, waiting time, value
3. Real life: dependencies, interruptions and thrashing

Priority inversion: an important task waits for an unimportant one

Setup (priority 1 = highest):

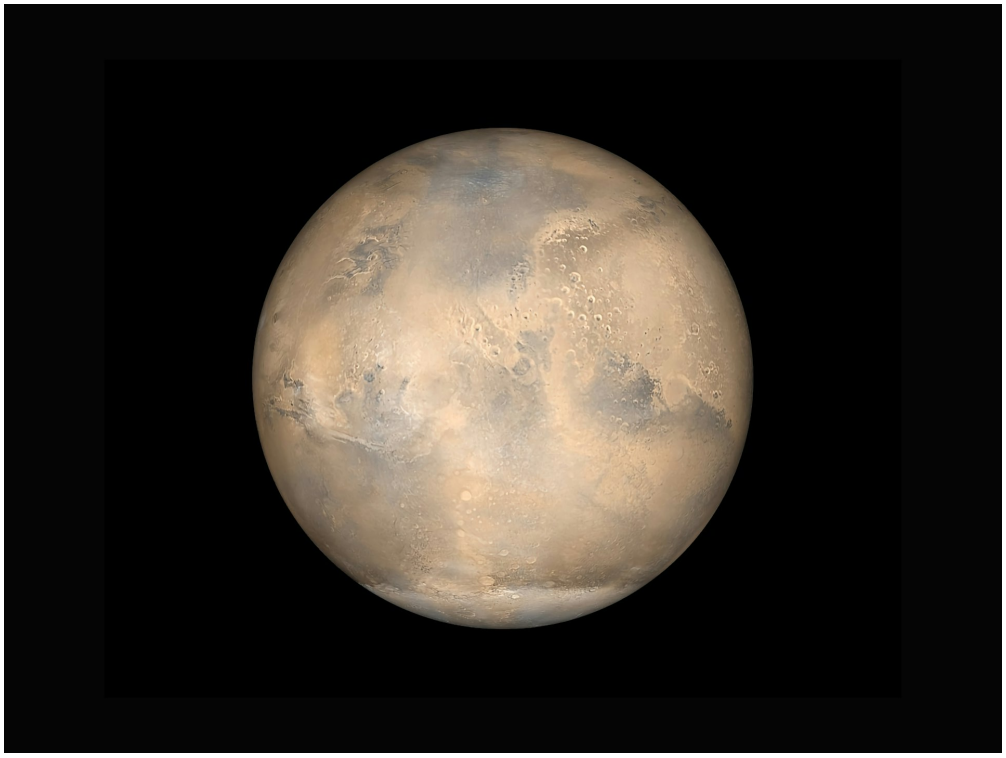
Task	Duration	Priority
A	20min	3
B	30min	1
C	30min	2
D	30min	2
E	30min	2



A must be finished before B can start

Challenge: High-priority tasks depend on low-priority tasks.

Risk: Priority inversion can lead to significant delays!



Priority inheritance: the blocker borrows the priority

Question: **How do we prevent the inversion?**

- Rule: Tasks inherit priority from their dependents.
- A inherits priority 1 from B and is done first
- B can start right after A, C to E follow

Task	Duration	Priority
A	20min	1

```
B    30min    1
C    30min    2
D    30min    2
E    30min    2
```

Interruptions: new tasks arrive while you work

- In reality we **cannot predict the future**
- New tasks arrive **while we work**
- Let's look at this for the earliest due date objective

A morning with seven tasks

8:00-12:00 Schedule, tasks arrive during the morning:

Task	Arrives	Duration	Deadline
Email A	8:00	20min	9:00
Create PPT	8:00	60min	10:50
Investor call	8:30	10min	9:00
Email B	8:40	30min	10:20
Liquidity	9:00	40min	11:00
Email C	9:00	20min	11:20
Email D	9:20	40min	10:00

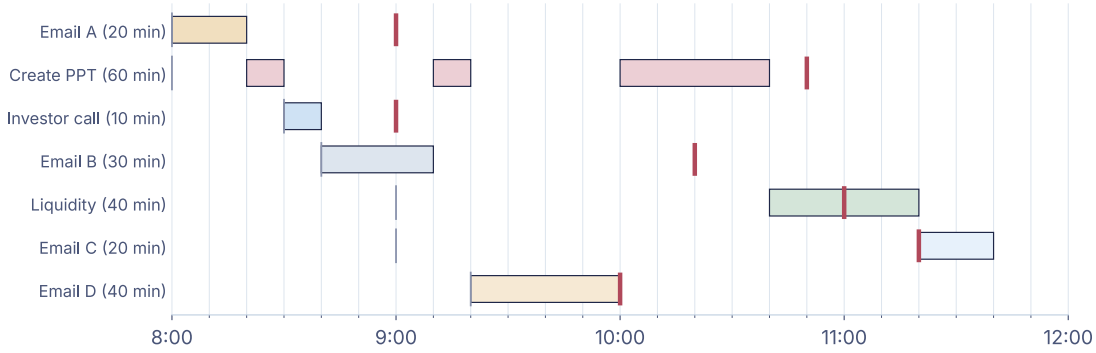
Question: **How would you start, if the goal is to avoid being late?**

EDD in real time: switch only for an earlier deadline

Rule:

- Always work on the task with the **earliest deadline**
- A new task with an **earlier** deadline interrupts the current one
- Otherwise, stick to the current task

The real-time EDD schedule



Grey line: arrival, red mark: deadline.

Question: **What's the maximum delay with this schedule?**

Same morning, new goal: as little waiting as possible

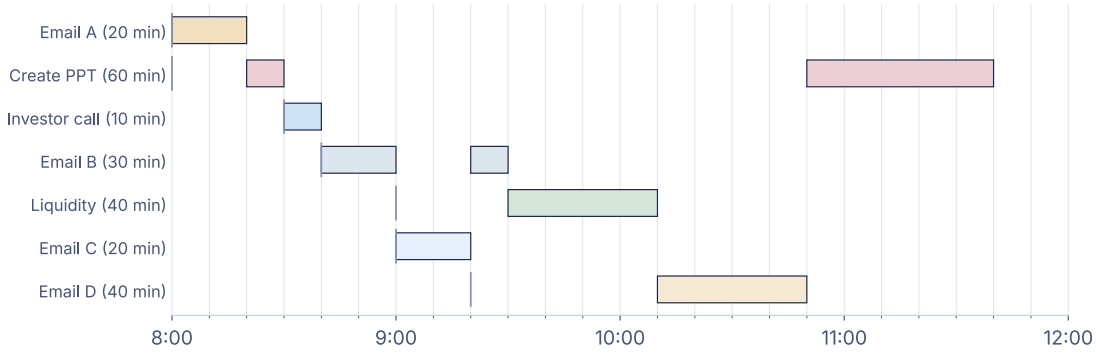
Question: **How would you start here?**

SPT in real time: switch only for a shorter task

Rule:

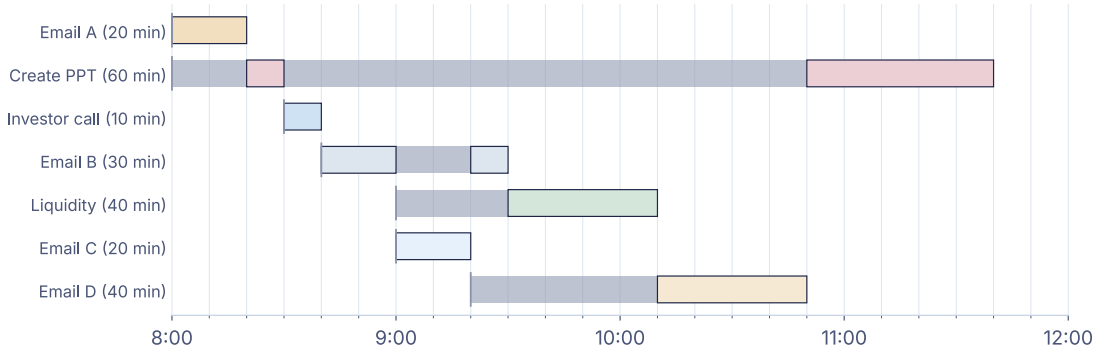
- Always work on the task with the **shortest duration**
- A new task that is **shorter** interrupts the current one
- Otherwise, stick to the current task

The real-time SPT schedule



Question: **Where can we see the waiting time?**

The real-time SPT schedule



Total waiting time: 260 minutes

Thrashing: switching so much that nothing gets done

- Excessive **context switching**
- Organization overhead exceeds productivity
- Maximum activity, minimum output

Question: **Have you ever experienced this?**

Warning signs of thrashing

- Constant **task switching**
- **Nothing** getting completed
- Increasing **stress levels**
- Declining **quality**

Question: **Any ideas how to prevent this?**

Preventing thrashing: long-term rules

1. Task rejection/delegation threshold
2. Simplified organization systems
3. Minimum work period rules
4. Reduced reactivity requirements

Preventing thrashing: today's tactics

1. Time blocking
2. Focus periods
3. Task batching
4. Priority freezes

Question: **What strategies have worked for you?**

How to use this starting today

- **Daily planning:** SPT for your to-do list, less waiting
- **Project management:** EDD when deadlines matter
- **Team coordination:** Johnson's rule for two-stage processes
- **Time blocking:** prevent thrashing

Real life: dependencies, interruptions and thrashing

What we covered

Dependencies make planning harder, and constant switching eats up your time. Protect blocks of focused work.

What we covered

The basics: when order matters, and when it doesn't

Sometimes the order of tasks changes the result, sometimes it doesn't. Knowing which saves time.

Your goal decides the rule: deadlines, waiting time, value

Each goal has its own simple rule: earliest deadline first to avoid being late, shortest task first to cut waiting, highest value per minute first to maximize what you deliver.

Real life: dependencies, interruptions and thrashing

Dependencies make planning harder, and constant switching eats up your time. Protect blocks of focused work.

Why it matters

Monday morning, ten tasks on your list: where do you start? Companies face the same question at scale: which order on the production line, which customer order first, which project gets the team this week.

First choose your goal.

Then choose your order.

Any questions

so far?

After the break: Scheduling

- Programming session in our new notebooks
- Today's rules in code: dictionaries and DataFrames
- Different scheduling algorithms applied to problems

Literature

Interesting literature to start

- Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.[1]

Books on Programming

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Free online
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

> **i** Note
>
> Think Python is free online and a great start. Schrödinger programmiert Python is a playful alternative in German.

[1] The main inspiration for this lecture. We have read it and discussed it in depth, always wanting to translate it into a course.

More Literature

For more interesting literature, take a look at the literature list of this course.